

EGE353

Autonomous Mobile Robotics

Service mobile robots, concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping.

Lab 5: Create ros package
Practical 1 (20%)

Chan Kit Wai
Tel: 65500559
Chan_kit_wai@nyp.edu.sg



Synopsis

This module aims to equip learners with the knowledge of service robots and their applications. Learners will also study the concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping. They will be able to implement mobile robotics system to carry out tasks autonomously.

Module Learning Outcomes

At the end of the module, learners are able to:

- Explain the features of different sensors and actuators used in autonomous mobile robotics.
- Perform the analysis of laser and vision sensor data using visualization tools.
- Implement mobile robotics system using methods such as mapping, localization, path planning, obstacle avoidance and motion control.

Create a workspace

A workspace is a directory containing ROS 2 packages. Before using ROS 2, it's necessary to source your ROS 2 installation workspace in the terminal you plan to work in. This makes ROS 2's packages available for you to use in that terminal.

You also have the option of sourcing an “overlay” – a secondary workspace where you can add new packages without interfering with the existing ROS 2 workspace that you're extending, or “underlay”. Your underlay must contain the dependencies of all the packages in your overlay. Packages in your overlay will override packages in the underlay. It's also possible to have several layers of underlays and overlays, with each successive overlay using the packages of its parent underlays.

**Task 1: Create a workspace and
clone a turtlesim to local drive**

Create a workspace

Start your terminal 1 and source the ros2 environment with the following command in your terminal. You may need to enter this command for each new terminal that you open.

```
$ source /opt/ros/humble/setup.bash
```

Create a new directory. For this example, it will be `dev_ws`. For other workspace it can be for example `sensor_ws` or `robotdog_ws`. Ensure you are in the correct directory using the `pwd` command as shown.

```
$ mkdir -p ~/dev_ws/src  
$ cd ~/dev_ws/src  
$ pwd  
/home/user/dev_ws/src  
$ ls
```

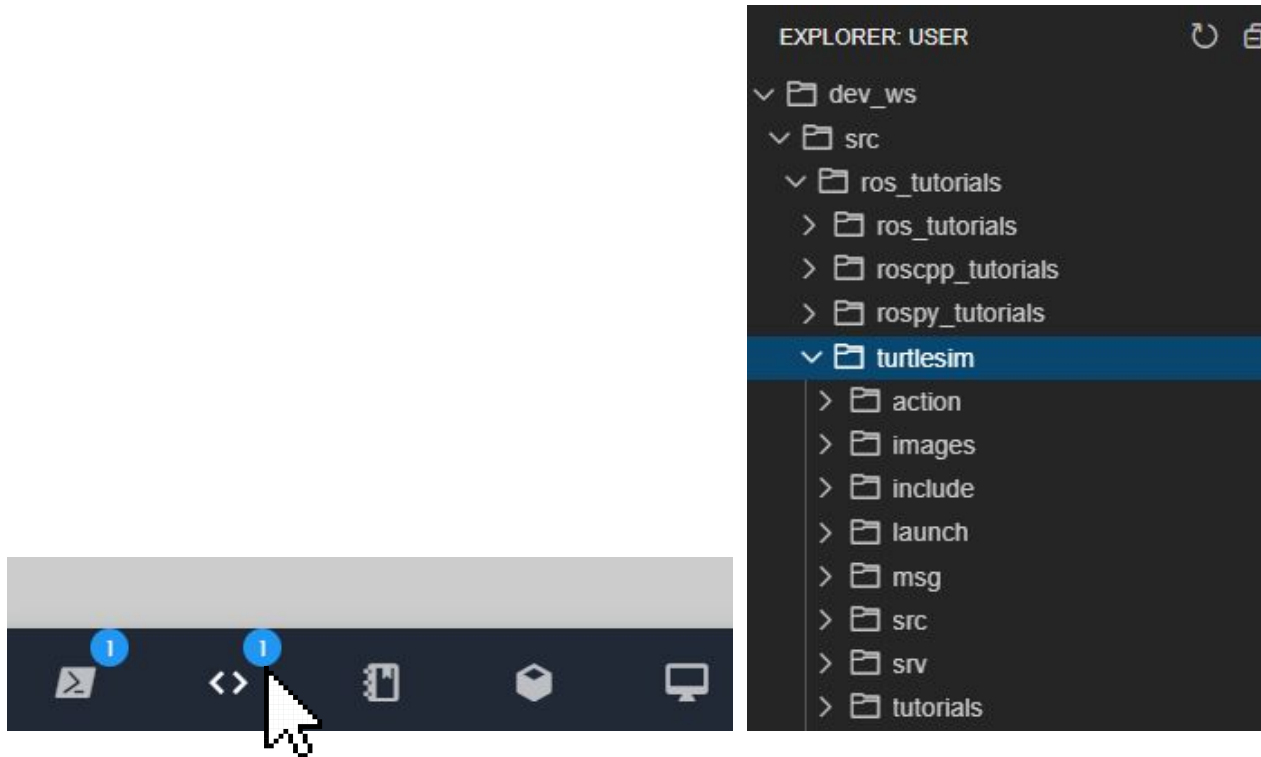
You will not see any files in the `src` directory. Ensure you're still in the `/home/user/dev_ws/src` directory before you clone. For now you will practice putting a workspace together using existing packages to be downloaded from github.com. But in the subsequent labs, you will create your own packages.

```
$ git clone https://github.com/ros/ros_tutorials.git -b humble  
$ ls  
ros_tutorials
```

Now `ros_tutorials` is cloned in your workspace (`/home/mobilerobot/dev_ws/src`). The `ros_tutorials` repository contains the `turtlesim` package, which we'll use in the rest of this tutorial. The other packages in this repository are not built because they contain a `COLCON_IGNORE` file. Now you have populated your workspace with a sample package, but it isn't a fully-functional workspace yet. You need to resolve dependencies and build the workspace first.

Create a workspace

Using code editor/file manager, explore the various files in the ros_tutorial directory.



Before building the workspace, you need to resolve package dependencies. You may have all the dependencies already, but best practice is to check for dependencies every time you clone. You wouldn't want a build to fail after a long wait because of missing dependencies.

```
~/dev_ws/src$ cd ..  
~/dev_ws$ rosdep install -i --from-path src --rosdistro humble -y  
#All required rosdeps installed successfully
```

Create a workspace

Using colcon build to generate other new directories. If you encounter errors, repeat colcon build for **another 3 to 4 times**.

```
~/dev_ws$ ls
src
~/dev_ws$ colcon build
...
This may be promoted to an error in a future release of colcon-core.
Starting >>> turtlesim
[Processing: turtlesim]
Finished <<< turtlesim [37.3s]

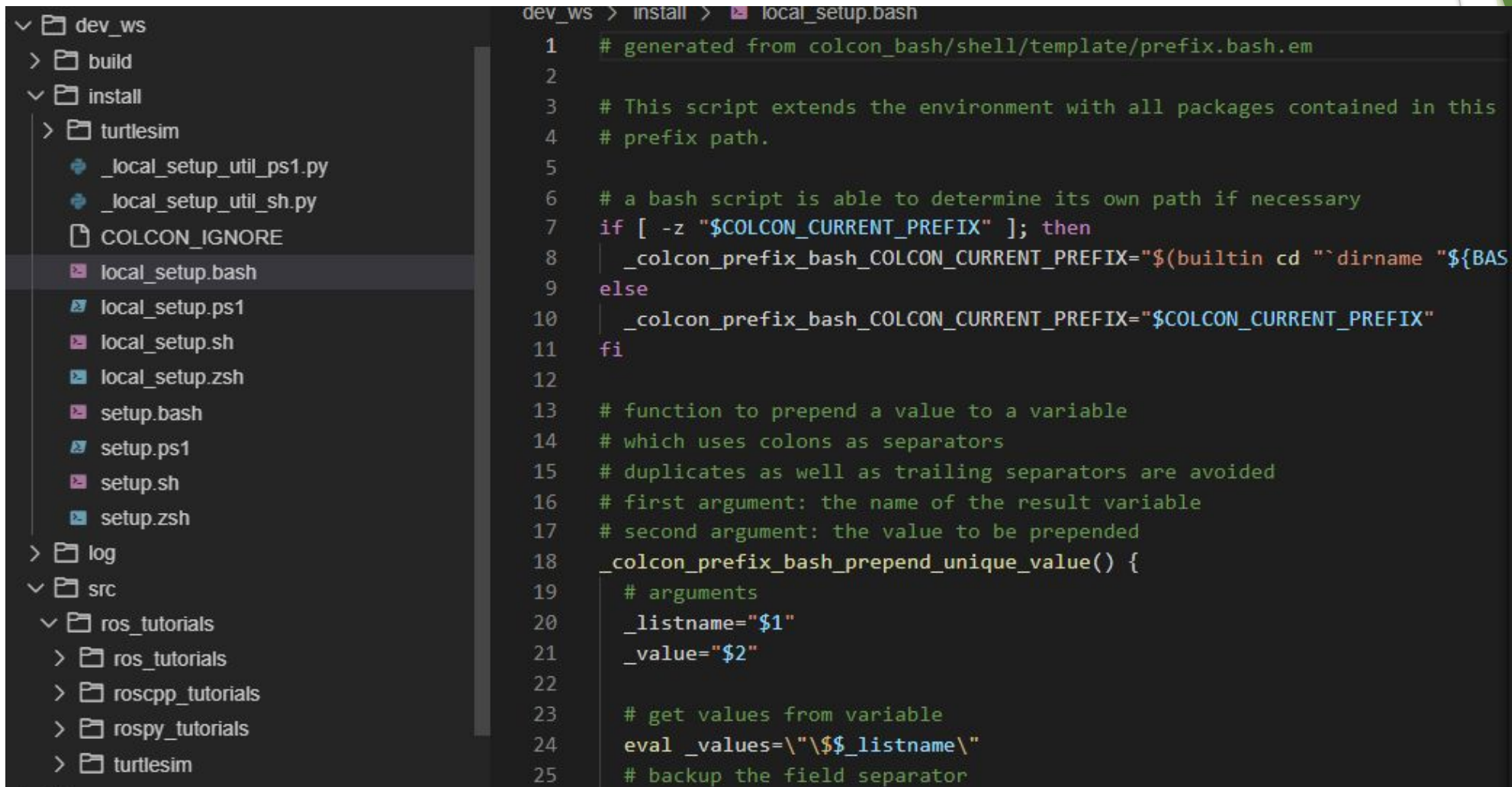
Summary: 1 package finished [37.4s]
~/dev_ws$ ls
build install log src
```

Using the file manager/code editor, open and explore the file in `install/local_setup.bash`. Do not edit or save the file. Sourcing the `local_setup` of the overlay will only add the packages available in the overlay to your environment. Setup sources the overlay as well as the underlay it was created in, allowing you to utilize both workspaces.

So, sourcing your main ROS 2 installation's setup and then the `dev_ws` overlay's `local_setup`, like you just did, is the same as just sourcing `dev_ws`'s setup, because that includes the environment of the underlay it was created in.

Create a workspace

Do not edit or save the local_setup.bash. Just browse through the content.



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the following structure:

- dev_ws
 - build
 - install
 - turtlesim
 - _local_setup_util_ps1.py
 - _local_setup_util_sh.py
 - COLCON_IGNORE
 - local_setup.bash (selected)
 - local_setup.ps1
 - local_setup.sh
 - local_setup.zsh
 - setup.bash
 - setup.ps1
 - setup.sh
 - setup.zsh
 - log
 - src
 - ros_tutorials
 - ros_tutorials
 - roscpp_tutorials
 - rospy_tutorials
 - turtlesim

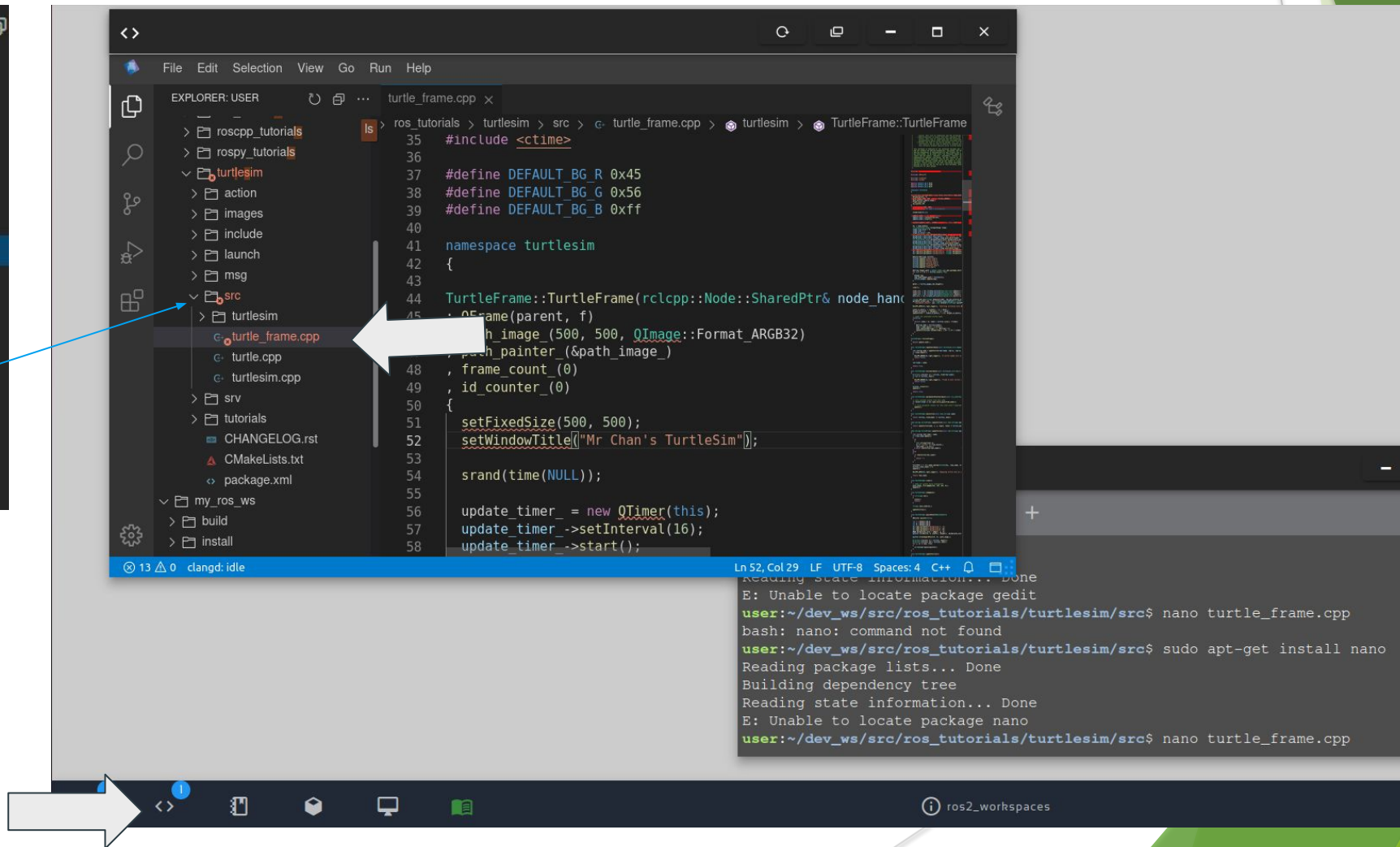
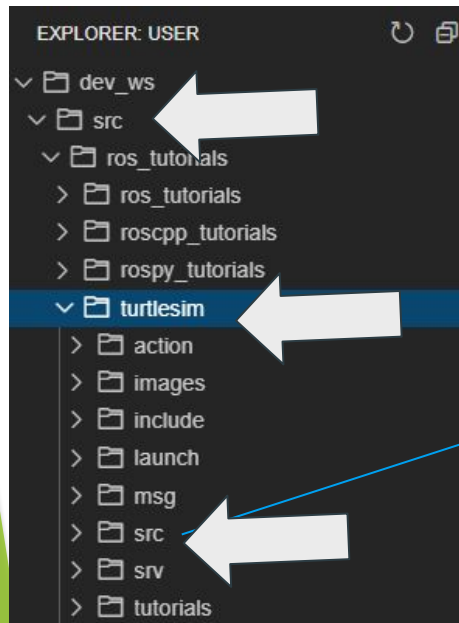
The code editor shows the content of the `local_setup.bash` file:

```
dev_ws > install > local_setup.bash
1  # generated from colcon_bash/shell/template/prefix.bash.em
2
3  # This script extends the environment with all packages contained in this
4  # prefix path.
5
6  # a bash script is able to determine its own path if necessary
7  if [ -z "$COLCON_CURRENT_PREFIX" ]; then
8    _colcon_prefix_bash_COLCON_CURRENT_PREFIX="$(builtin cd "`dirname "${BAS
9  else
10   _colcon_prefix_bash_COLCON_CURRENT_PREFIX="$COLCON_CURRENT_PREFIX"
11 fi
12
13 # function to prepend a value to a variable
14 # which uses colons as separators
15 # duplicates as well as trailing separators are avoided
16 # first argument: the name of the result variable
17 # second argument: the value to be prepended
18 _colcon_prefix_bash_prepend_unique_value() {
19   # arguments
20   _listname="$1"
21   _value="$2"
22
23   # get values from variable
24   eval _values="\${$_listname}"
25   # backup the field separator
```


Create a workspace

Modify the turtlesim code. Open the file turtle_frame.cpp.

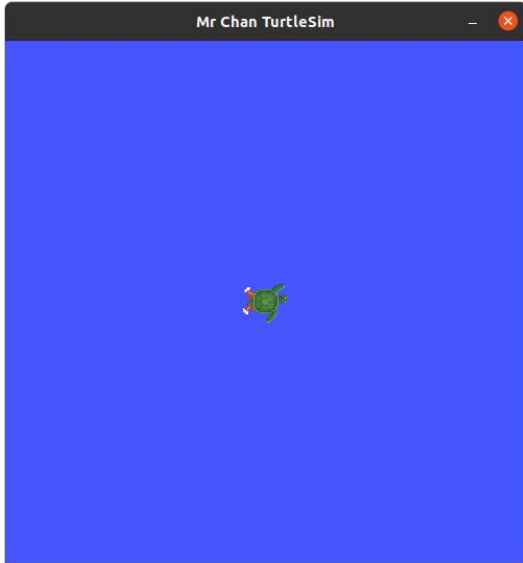
Modify line 52: `setWindowTitle("Mr Chan TurtleSim");`



Create a workspace

Save the file `turtle_frame.cpp`. Go back to the 1st terminal and rebuild the files. You need to rebuild each time you modify the source code. The `source /opt/ros/humble/setup.bash`: Activates the core ROS installation. The `source install/local_setup.bash`: Activates your custom workspace's packages, overlaying them on the core ROS installation.

```
~/dev_ws$ colcon build
~/dev_ws$ source install/local_setup.bash
~/dev_ws$ ros2 run turtlesim turtlesim_node
```



`source /opt/ros/humble/setup.bash`

System turtlesim
(install with ros)

`source install/local_setup.bash`

local turtlesim
(source code compiled in dev_ws)

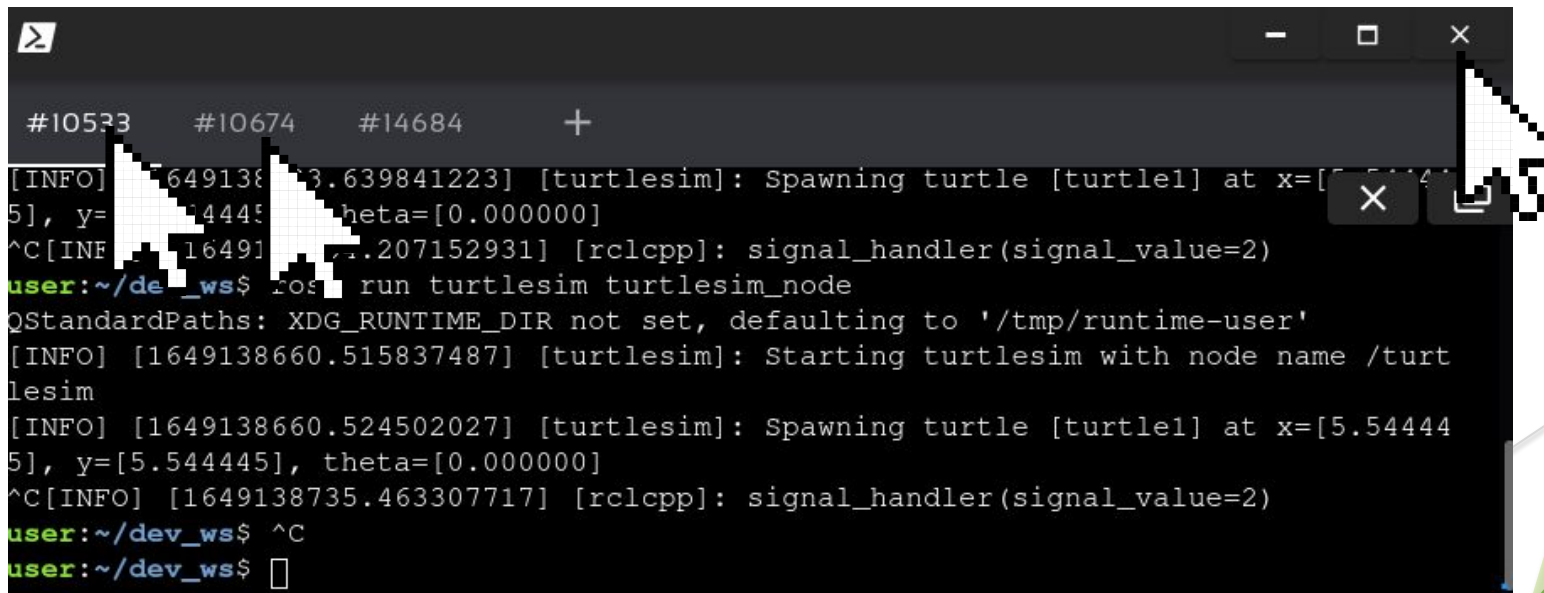
Enlarge and take note that the title is changed to your name i.e Mr Chan's Turtlesim. This is

Create a workspace

Open a 2nd terminal.

```
$ source /opt/ros/humble/setup.bash  
$ ros2 run turtlesim turtlesim_node
```

CTR+C on each terminal to stop each process.



The screenshot shows a terminal window with a dark background and light text. At the top, there are window control buttons (minimize, maximize, close) and a tab bar with three tabs labeled #10533, #10674, and #14684, followed by a '+' button. The terminal output shows the following sequence of events:

```
#10533 #10674 #14684 +  
[INFO] [1649138660.515837487] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]  
^C[INFO] [1649138735.463307717] [rclcpp]: signal_handler(signal_value=2)  
user:~/dev_ws$ ros2 run turtlesim turtlesim_node  
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-user'  
[INFO] [1649138660.515837487] [turtlesim]: Starting turtlesim with node name /turtlesim  
[INFO] [1649138660.524502027] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]  
^C[INFO] [1649138735.463307717] [rclcpp]: signal_handler(signal_value=2)  
user:~/dev_ws$ ^C  
user:~/dev_ws$
```

Task 2: Create a ROS2 package

Create Ros2 package

A package can be considered a container for your ROS 2 code. If you want to be able to install your code or share it with others, then you'll need it organized in a package. With packages, you can release your ROS 2 work and allow others to build and use it easily.

Package creation in ROS 2 uses ament as its build system and colcon as its build tool. You can create a package using either CMake or Python, which are officially supported, though other build types do exist.

- package.xml file containing meta information about the package
- CMakeLists.txt file that describes how to build the code within the package

The simplest possible package may have a file structure that looks like:

`my_package/`

`CMakeLists.txt`

`package.xml`

Create Ros2 package

A single workspace can contain as many packages as you want, each in their own folder. You can also have packages of different build types in one workspace (CMake, Python, etc.). You cannot have nested packages.

Best practice is to have a src folder within your workspace, and to create your packages in there. This keeps the top level of the workspace “clean”.

A trivial workspace might look like:

```
workspace_folder/  
  src/  
    package_1/  
      CMakeLists.txt  
      package.xml  
  
    package_2/  
      setup.py  
      package.xml  
      resource/package_2  
  
    ...  
    package_n/  
      CMakeLists.txt  
      package.xml
```

Create Ros2 package

Click on terminal 1 and source your workspace

```
~/dev_ws$ source install/local_setup.bash
```

Make sure you are in the src folder before running the package creation command. You will now have a new folder within your workspace's src directory called my_package.

To create a package use the template: `ros2 pkg create --build-type ament_cmake <package_name>`

```
~/dev_ws$ cd src
~/dev_ws/src$ ls
ros_tutorials
~/dev_ws/src$ ros2 pkg create --build-type ament_cmake --node-name my_node my_package
creating folder ./my_package
creating ./my_package/package.xml
creating source and include folder
creating folder ./my_package/src
creating folder ./my_package/include/my_package
creating ./my_package/CMakeLists.txt
creating ./my_package/src/my_node.cpp
~/dev_ws/src$ ls
my_package  ros_tutorials
~/dev_ws/src$ cd ..
~/dev_ws$ colcon build
~/dev_ws$ ls \build
COLCON_IGNORE  my_package  turtlesim
```

Create Ros2 package

if you want to build any particular package only, use “colcon build --packages-select my_package” command.

To use your new package and executable, source your main ROS 2 installation.

```
~/dev_ws$ source install/setup.bash
~/dev_ws$ ros2 run my_package my_node
hello world my_package package
```

```
~/dev_ws$ ls src/my_package/
CMakeLists.txt include package.xml src
~/dev_ws$ ls src/my_package/src
my_node.cpp
```

my_node.cpp is inside the src directory. This is where all your custom C++ nodes will go in the future.

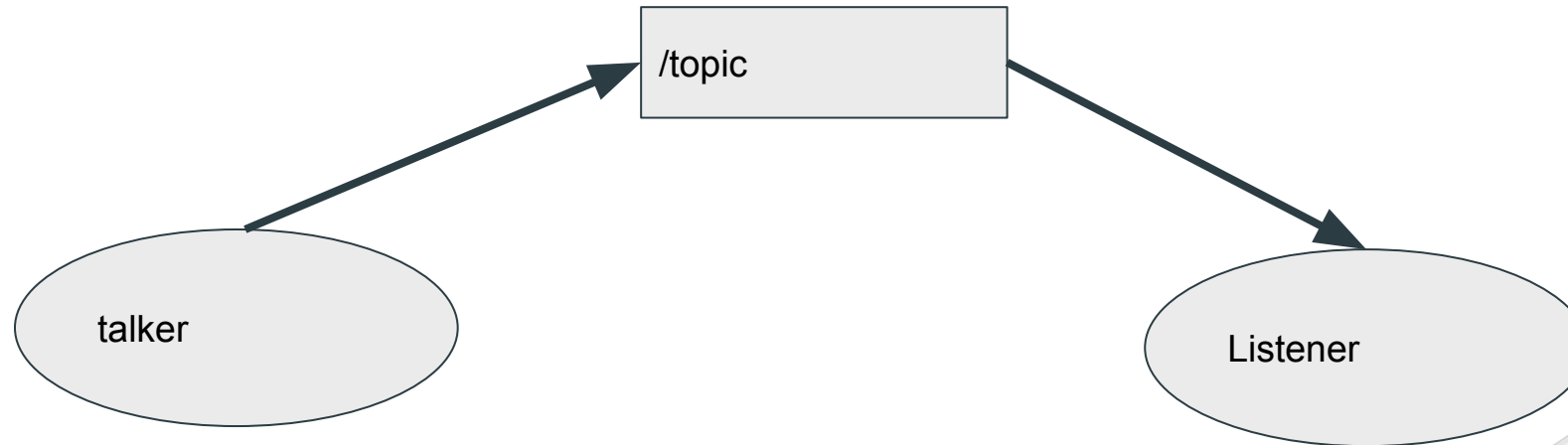
Customize package.xml

You may have noticed in the return message after creating your package that the fields description and license contain TODO notes. That’s because the package description and license declaration are not automatically set, but are required if you ever want to release your package. The maintainer field may also need to be filled in.

From dev_ws/src/my_package, open package.xml using file manager/code editor:

Modify line 6 and save: <description>**Your full name** first ros package</description>

Task 3: Create a Publisher and Subscribe package



Simple publisher and subscriber

Write the publisher node

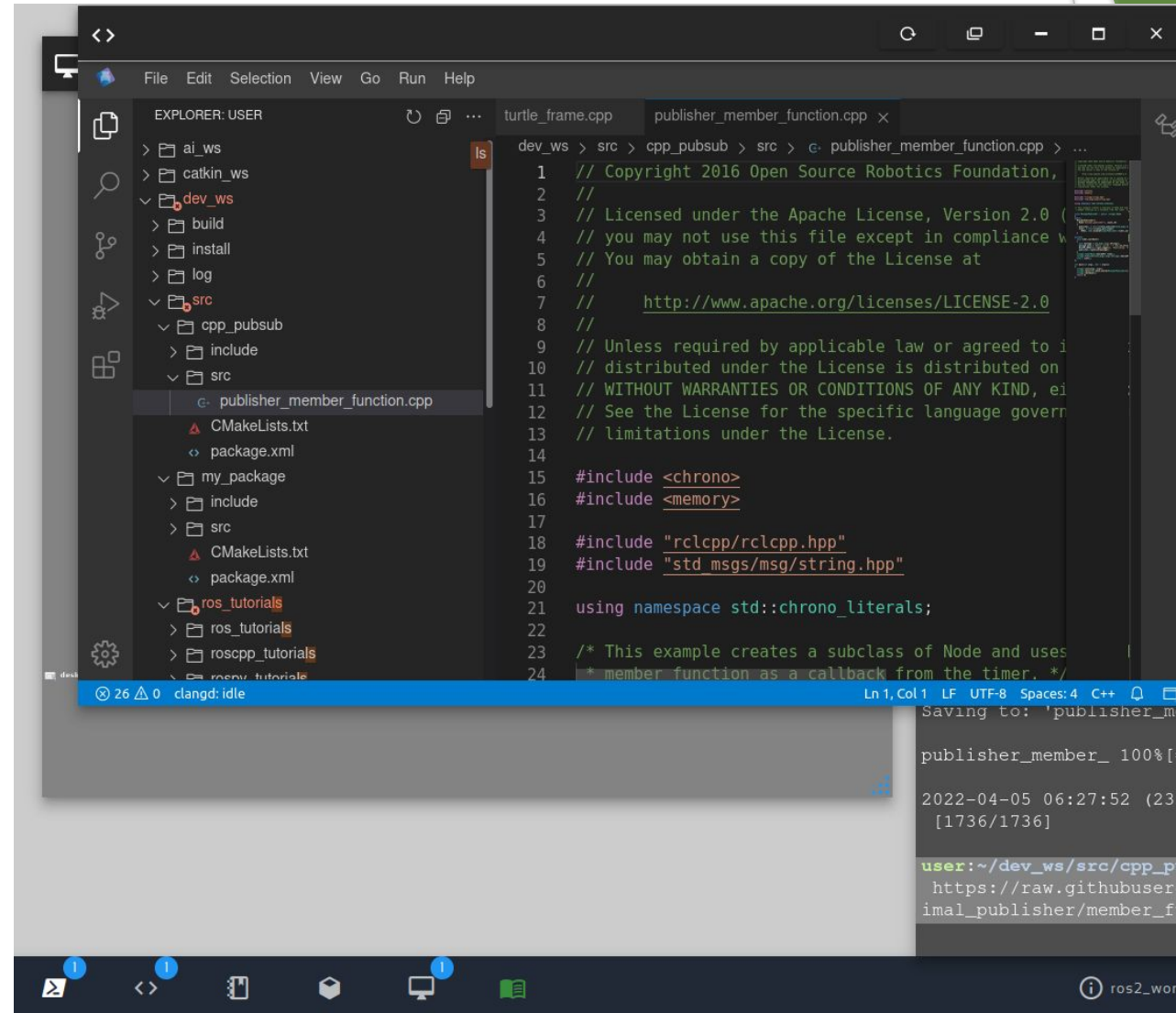
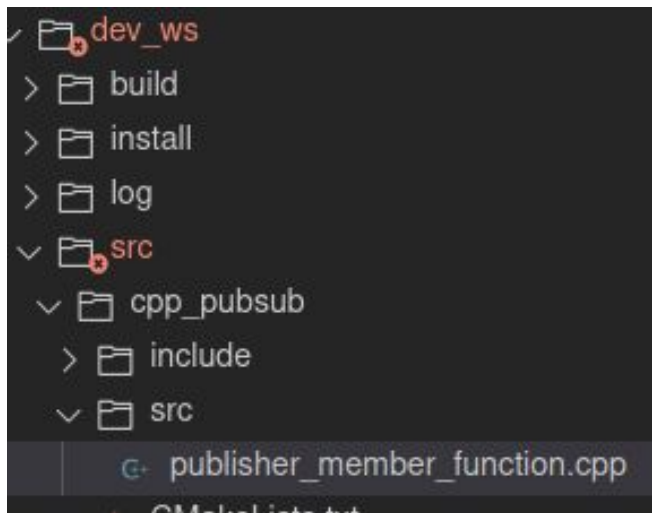
Nodes are executable processes that communicate over the ROS graph. In this tutorial, the nodes will pass information in the form of string messages to each other over a topic. The example used here is a simple “talker” and “listener” system; one node publishes data and the other subscribes to the topic so it can receive that data.

You will download publisher_member_function.cpp to src folder.

```
~/dev_ws$ cd src
~/dev_ws/src$ ros2 pkg create --build-type ament_cmake cpp_pubsub
....
creating ./cpp_pubsub/CMakeLists.txt
~/dev_ws/src$ ls
cpp_pubsub  my_package  ros_tutorials
~/dev_ws/src$ cd cpp_pubsub/src
~/dev_ws/src/cpp_pubsub/src$ ls
~/dev_ws/src/cpp_pubsub/src$
~/src$ wget -O publisher_member_function.cpp https://raw.githubusercontent.com/ros2/examples/humble/rclcpp/topics/minimal_publisher/member_function.cpp
...
~/dev_ws/src/cpp_pubsub/src$ ls
publisher_member_function.cpp
```

Simple publisher and subscriber

Open the file publisher_member_function.cpp in IDE



Simple publisher and subscriber

```
#include <chrono>
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
```

```
using namespace std::chrono_literals;
```

```
class MinimalPublisher : public rclcpp::Node
```

```
{
public:
    MinimalPublisher()
    : Node("minimal_publisher"), count_(0)
    {
        publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
        timer_ = this->create_wall_timer(
            500ms, std::bind(&MinimalPublisher::timer_callback, this));
    }
}
```

```
private:
```

```
void timer_callback()
{
    auto message = std_msgs::msg::String();
    message.data = "Hello, world! " + std::to_string(count_++);
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);
}
rclcpp::TimerBase::SharedPtr timer_;
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
size_t count_;
};
```

```
int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalPublisher>());
    rclcpp::shutdown();
    return 0;
}
```

The top of the code includes the standard C++ headers you will be using. After the standard C++ headers is the `rclcpp/rclcpp.hpp` include which allows you to use the most common pieces of the ROS 2 system. Last is `std_msgs/msg/string.hpp`, which includes the built-in message type you will use to publish data.

These lines represent the node's dependencies. Recall that dependencies have to be added to `package.xml` and `CMakeLists.txt`, which you'll do in the next section.

The top of the code includes the standard C++ headers you will be using. After the standard C++ headers is the `rclcpp/rclcpp.hpp` include which allows you to use the most common pieces of the ROS 2 system. Last is `std_msgs/msg/string.hpp`, which includes the built-in message type you will use to publish data.

These lines represent the node's dependencies. Recall that dependencies have to be added to `package.xml` and `CMakeLists.txt`, which you'll do in the next section.

The public constructor names the node `minimal_publisher` and initializes `count_` to 0. Inside the constructor, the publisher is initialized with the `String` message type, the topic name `topic`, and the required queue size to limit messages in the event of a backup. Next, `timer_` is initialized, which causes the `timer_callback` function to be executed twice a second.

The `timer_callback` function is where the message data is set and the messages are actually published. The `RCLCPP_INFO` macro ensures every published message is printed to the console.

Following the `MinimalPublisher` class is `main`, where the node actually executes. `rclcpp::init` initializes ROS 2, and `rclcpp::spin` starts processing data from the node, including callbacks from the timer.

Simple publisher and subscriber

Package.xml

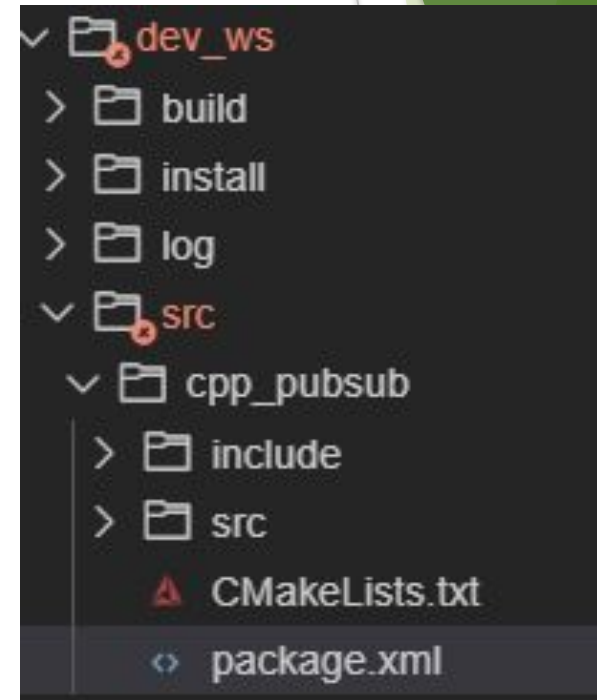
Open the file `~/dev_ws/src/cpp_pubsub/package.xml` in code editor and add in the dependencies. Save the file.

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>cpp_pubsub</name>
  <version>0.0.0</version>
  <description>My first publisher and subscriber</description>
  <maintainer email="mobilerobot@todo.todo">mobilerobot</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
    <depend>rclcpp</depend>
    <depend>std_msgs</depend>
  </export>
</package>
```



Simple publisher and subscriber

CMakeList.txt

Open CMakeList.txt and add in the required dependency, packages and source files. The file CMakeLists.txt is the input to the CMake build system for building software packages. Any CMake-compliant package contains one or more CMakeLists.txt file that describe how to build the code and where to install it to. The CMakeLists.txt file used for a catkin project is a standard vanilla CMakeLists.txt file with a few additional constraints.

```
~/dev_ws/src/cpp_pubsub/src$ cd ..
```

Open the file ~/dev_ws/src/cpp_pubsub\$/CMakeList.txt in IDE/code editor

Simple publisher and subscriber

CMakeList.txt Open the file `~/dev_ws/src/cpp_pubsub$/CMakeList.txt` in IDE/code editor, edit and save it.

```
cmake_minimum_required(VERSION 3.5)
project(cpp_pubsub)

# Default to C99
if(NOT CMAKE_C_STANDARD)
  set(CMAKE_C_STANDARD 99)
endif()

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # uncomment the line when a copyright and license is not present in all source files
  #set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # uncomment the line when this package is not in a git repo
  #set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

Simple publisher and subscriber

Write the subscriber node

```
~/dev_ws/src/cpp_pubsub$ cd src
~/dev_ws/src/cpp_pubsub/src$ ls
publisher_member_function.cpp
~/src$ wget -O subscriber_member_function.cpp https://raw.githubusercontent.com/ros2/examples/humble/rclcpp/topics/minimal\_subscriber/member\_function.cpp
--2022-02-28 03:59:55-- https://raw.githubusercontent.com/ros2/examples/foxy/rclcpp/topics/minimal\_subscriber/member\_function.cpp
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address 'raw.githubusercontent.com'
~/dev_ws/src/cpp_pubsub/src$ ls
publisher_member_function.cpp subscriber_member_function.cpp
~/dev_ws/src/cpp_pubsub/src$ cd ~/dev_ws
```

The subscriber node's code is nearly identical to the publisher's. Now the node is named `minimal_subscriber`, and the constructor uses the node's `create_subscription` class to execute the callback.

There is no timer because the subscriber simply responds whenever data is published to the topic topic.

Simple publisher and subscriber

View the subscriber node, add in your name in the topic callback function and save the file

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }

private:
    void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
    {
        RCLCPP_INFO(this->get_logger(), "My name is ???, I heard: '%s'",
            msg->data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

Recall from that the topic name and message type used by the publisher and subscriber must match to allow them to communicate.

The topic_callback function receives the string message data published over the topic, and simply writes it to the console using the RCLCPP_INFO macro.

The only field declaration in this class is the subscription.

The main function is exactly the same, except now it spins the MinimalSubscriber node. For the publisher node, spinning meant starting the timer, but for the subscriber it simply means preparing to receive messages whenever they come.

Since this node has the same dependencies as the publisher node, there's nothing new to add to package.xml.

Simple publisher and subscriber

Edit CMakeList.txt and save

```
cmake_minimum_required(VERSION 3.5)
project(cpp_pubsub)

# Default to C99
if(NOT CMAKE_C_STANDARD)
  set(CMAKE_C_STANDARD 99)
endif()

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  talker
  listener
  DESTINATION lib/${PROJECT_NAME})

.....
```

Simple publisher and subscriber

Build and Run

Use the first terminal to run the talker node which is the publisher.

```
~/dev_ws$ colcon build --packages-select cpp_pubsub
```

```
Starting >>> cpp_pubsub
```

```
Finished <<< cpp_pubsub [0.21s]
```

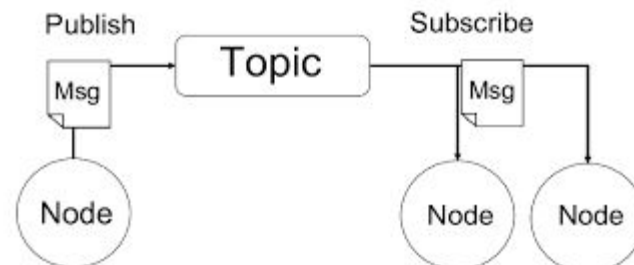
```
Summary: 1 package finished [0.30s]
```

```
~/dev_ws$ source install/setup.bash
```

```
~/dev_ws$ ros2 run cpp_pubsub talker
```

```
[INFO] [1646051316.464444988] [minimal_publisher]: Publishing: 'Hello, world! 0'
```

```
[INFO] [1646051316.964443538] [minimal_publisher]: Publishing: 'Hello, world! 1'
```



Simple publisher and subscriber

Build and Run

Open a 2nd terminal for the listener which is the subscriber.

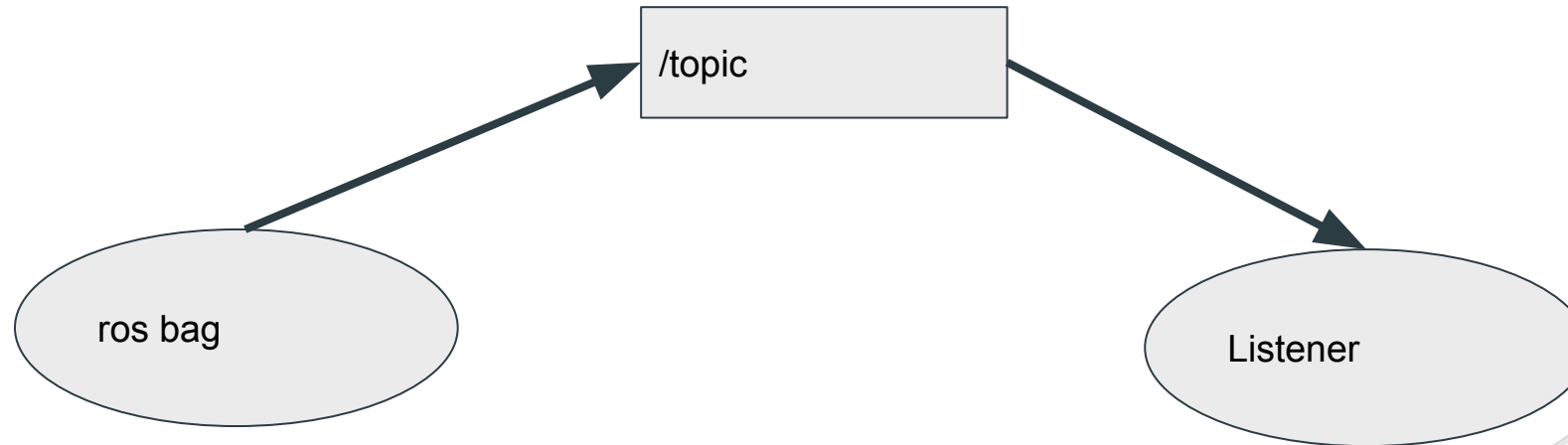
```
~/dev_ws$ source install/setup.bash
```

```
~/dev_ws$ ros2 run cpp_pubsub listener
```

```
[INFO] [1646051746.535122648] [minimal_subscriber]: I heard: 'Hello, world! 0'
```

```
[INFO] [1646051747.034727298] [minimal_subscriber]: I heard: 'Hello, world! 1'
```

Task 4: Create a rosbag Publisher and Subscriber package



Practical 1 (20%)

You will need to refer to previous lab for the following task. Using rosbag to record the topic into a bag file and use the bag file to publish the topic with the listener subscribing the topic. Open a new terminal. `cd ..` or `cd bag_files` to go to `bag_files` directory and record the topic into `bagtalker.bag`.

Run only `ros bag play bagtalker.bag` in a loop and listener node. Run the `rqt_graph`.

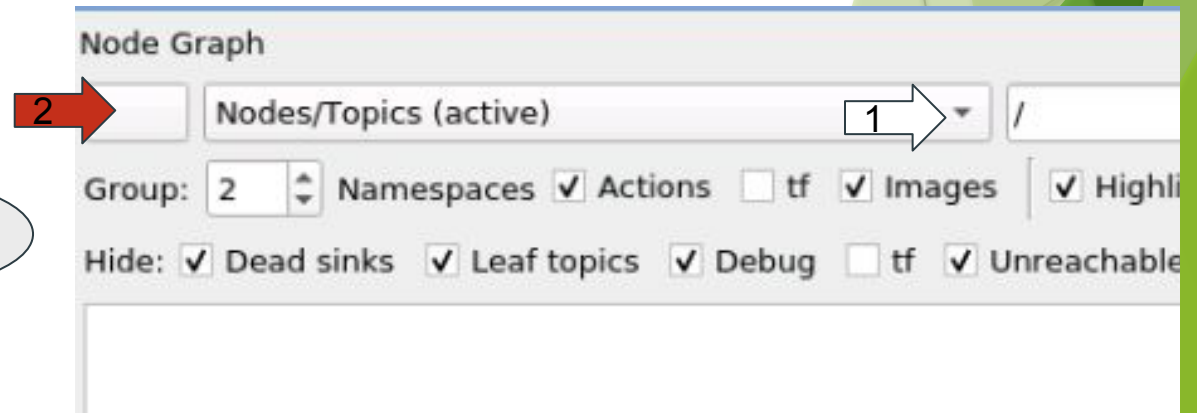
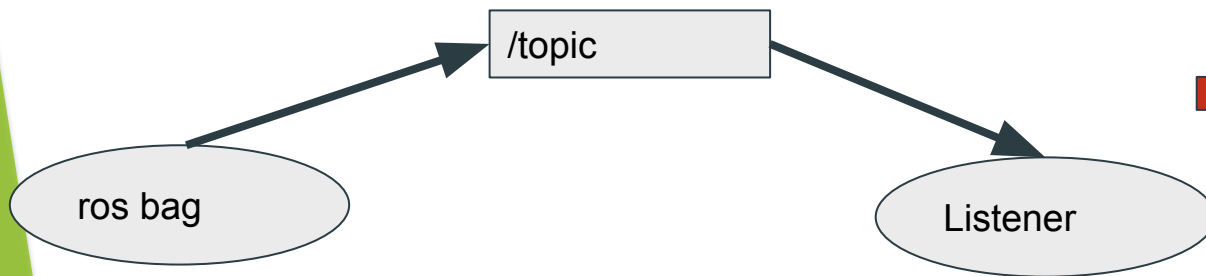
Demo the following

Close all previous terminal

Terminal 1 -> run bag file

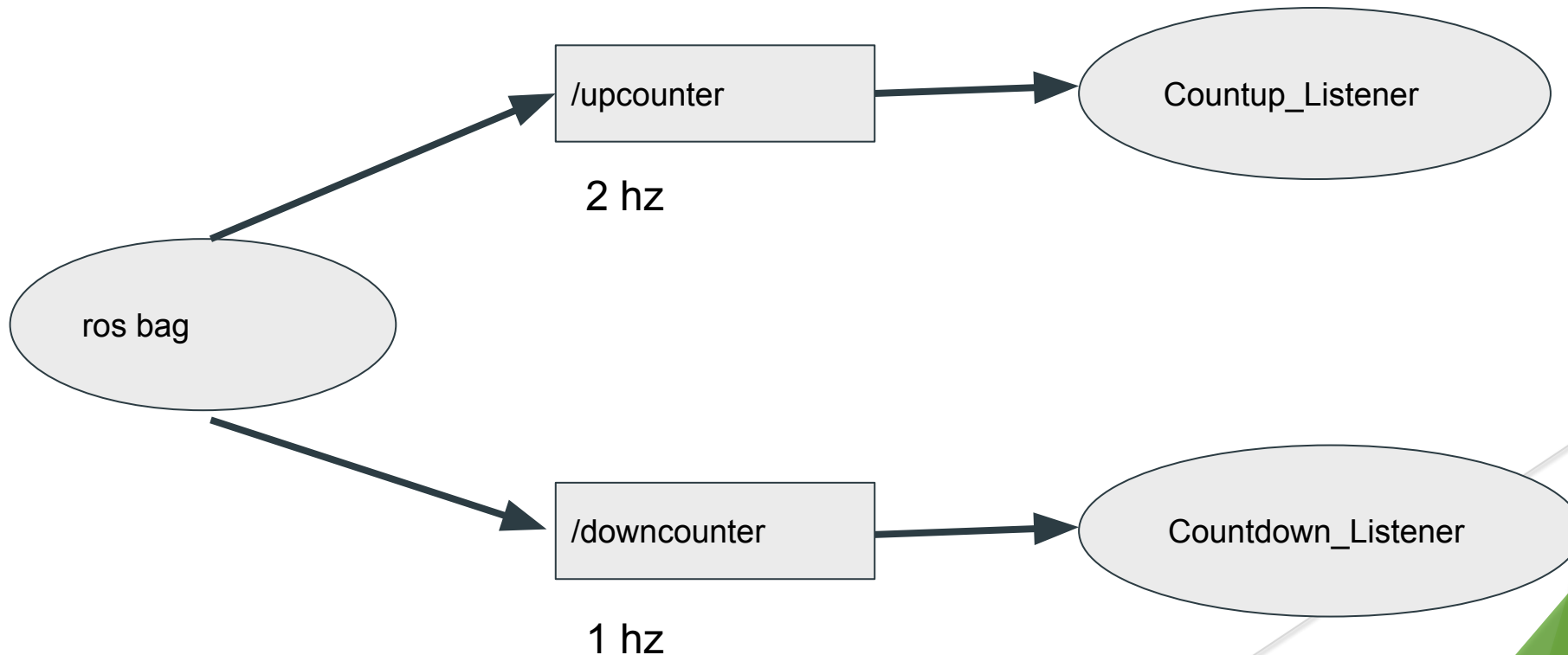
Terminal 2 -> run listener

Terminal 3 -> `ros2 run rqt_graph rqt_graph` -> click node/topic (active)
-> click refresh button



Task 5: Create a rosbag Publisher and 2 Subscriber package (optional)

This task is for independent/advance learners with minimum supervision.



Practical 1 (20%)

Individual work.

Live demo to instructor is compulsory, questions will be asked during the assessment.
Demo the following task upon completion

Task4

Task5

Late demo will be subjected to penalty mentioned in the Politemall -> module overview

- Note the difference between running turtlesim from terminal 1 and terminal 2. Explain what is the difference and the reason behind it.
- Explain what is git clone ?
- When do you need to run colcon build command ?
- What is the name of the topic that the talker is publishing ?
- What is the name of the publisher and subscriber node ?
- What is the command to view the frequency of the topic, what is the freq ?
- What is the message type of the topic ?
- Give some other examples of listener and talker node used in robotic system. (i.e. camera sensor is talker node)

Demo to instructor is compulsory

Prepare to answer and screenshot your results to a google doc file for lab demo.

Question asked may not be limited to the above questions.

End of briefing for Lab session